LettuceDetect Reimplementation: Project Report

Sem 5 Minor Research Project Paper reimplemented: LettuceDetect: A Hallucination Detection Framework for RAG Applications (Kovács & Recski, Feb 2025, arXiv:2502.17125) **Dataset:** RAGTruth (Niu et al., ACL 2024) **Repository:** github.com/AfzalRaja001/lettucedetect-reimpl (private)

This document is a self-contained, start-to-end account of the project: what we set out to do, what we built, what we tried, what worked, what didn't, and what we can honestly conclude. It is written to be used directly as source material for the final presentation.

Table of Contents

1. Executive Summary
2. Problem Statement
3. Related Work: Summary
4. The Paper We Reimplemented
5. Implementation
6. Results
7. What Worked
8. What Didn't Work / Challenges Encountered
9. Conclusions
10. Limitations
11. Future Work
12. References

1. Executive Summary

We built a from-scratch reimplementation of **LettuceDetect**, a token-level hallucination detector for Retrieval-Augmented Generation (RAG) systems, trained and evaluated on the full **RAGTruth** benchmark. Our reimplementation achieved **77.18 example-level F1**, matching and slightly exceeding the paper's own published result for LettuceDetect-base (**76.07**), on an independently written codebase. We did not call the authors' pip package.

Beyond reproduction, we held our own work to the same standard of scrutiny the recent literature applies to this whole class of detector. We identified and disclosed a validation leak in our own checkpoint-selection methodology, built an honest held-out protocol for post-hoc threshold tuning rather than reporting whichever number looked best, and verified our single most error-prone implementation step (tokenization and label alignment) against the official reference implementation before trusting it on the full dataset. We also found that a "beyond the paper" idea we initially considered, training a detector on the newer PsiloQA dataset and reporting the numbers, had already been done in PsiloQA's own paper. We pivoted to the actual open question that work leaves unanswered: whether training on RAGTruth and PsiloQA *together* closes the cross-dataset generalization gap that paper reports. That combined-training study is fully scoped and ready to execute as the next phase of this project.

2. Problem Statement

Retrieval-Augmented Generation (RAG) hands a large language model (LLM) retrieved passages, the **context**, alongside a question, so it answers from evidence rather than relying purely on frozen internal memory. This is the dominant architecture for production LLM systems needing current, private, or verifiable information.

The failure this project addresses: **even when the retrieved context is correct and sufficient, an LLM's answer can still contain claims the context does not support.** This is a **hallucination**, defined precisely as *unsupported by the given context*, not "false in the real world." A true statement the context never mentioned still counts, because the model had no license to assert it from what it was given. This narrow definition is what makes the problem checkable by a program at all: verifying a claim against a fixed document never requires leaving the room.

Prior detectors split into two unsatisfying families. **LLM-as-judge** approaches are accurate, but expensive, doubling or tripling inference cost at production scale. **Small encoder-based classifiers** are cheap, but built on BERT-family backbones capped at 512 tokens, physically unable to see realistic 2,000 to 8,000-token RAG contexts. LettuceDetect's contribution is recognizing that a long-context encoder (ModernBERT, up to 8,192 tokens) finally makes the "classify every answer token" approach viable at a size and cost neither prior family could match.

3. Related Work: Summary

Condensed:

Method	Year	Approach	Key limitation
SelfCheckGPT	2023	Sample-consistency, no context used	Expensive (N samples), not RAG-native
RAGTruth	2024	*(the benchmark, not a detector)*	English-only, no official span metric
Luna	2024	DeBERTa-large token classifier	Still short-context; larger than LettuceDetect
RAG-HAT	2024	8B LLM detector + DPO correction	Heaviest pipeline; highest F1 (83.9)
ReDeEP	2024	Reads generator's internal attention/FFN	White-box only; needs generator access
LettuceDetect	**2025**	**ModernBERT token classifier**	**English-only; no calibration; untested OOD generalization**
Turk-LettuceDetect	2025	Same recipe, translated to Turkish	F1 drop to 72.7 from translation noise
Dubanowska et al.	2025	OOD-generalization stress test	Diagnostic only, no fix proposed
PsiloQA	2025	14-language, LLM-annotated span data	No human verification of labels
RAGognizer	2026	Detection head trained into the generator	Needs to control/retrain the generator

The throughline: each entry reacts to a specific limitation in the one before it. SelfCheckGPT's cost problem leads to Luna's context-window problem, which leads to LettuceDetect's answer to that, and the field then fans out into generalization, multilinguality, and integration questions. Our project sits at exactly the generalization fork: LettuceDetect solves the cost/context problem, and whether it (or any detector like it) actually generalizes is the open question we target in section 11.

4. The Paper We Reimplemented

Core idea: reframe hallucination detection as **token-level binary classification**. Given `(context, question, answer)`, classify every token of the answer as `0` (supported) or `1` (hallucinated), rather than producing one label for the whole answer.

Why token-level: first, more supervision per example. A 300-token answer yields 300 training signals instead of one. Second, it localizes the problem, since a user can be shown exactly which phrase is unsupported. Third, the coarse yes/no verdict falls out for free by taking the logical OR across token predictions.

The enabler, ModernBERT: original BERT caps at 512 tokens because it learns one position vector per slot from 1 to 512. ModernBERT (AnswerDotAI, Dec 2024) replaces this with **RoPE** (Rotary Position Embeddings), encoding position as a mathematical rotation defined for any position. That removes the ceiling and extends capacity to **8,192 tokens**. This is the paper's single key technical enabler: token-level hallucination classification was not a new idea, it simply was not *possible* at this size and cost before a long-context encoder existed.

Architecture:

```
[CLS] Context [SEP] Answer [SEP]
                ↓
     ModernBERT backbone (base: 150M params)
                ↓
     Linear token-classification head → 2 classes per token
                ↓
     Per-token P(hallucinated); threshold at 0.5
                ↓
     Merge consecutive flagged tokens → character-level spans
```

Context tokens are labeled `-100` (excluded from the loss); only answer tokens are graded `0` / `1`.

Published training recipe: AdamW, learning rate 1e-5, weight decay 0.01, 6 epochs, batch size 8, max sequence length 4,096, on an A100 GPU.

Published results (RAGTruth, example-level F1):

Method	F1
GPT-3.5-turbo prompting	52.9
SelfCheckGPT	58.8
Luna	65.4
GPT-4-turbo prompting	63.4
Fine-tuned Llama-2-13B	78.7
RAG-HAT	83.9
LettuceDetect-base	**76.07**
LettuceDetect-large	79.22

5. Implementation

5.1 Development and Training Workflow

Training a 150M-parameter model on roughly 15,000 examples for 6 epochs needs a real GPU. Developing and debugging the pipeline does not. We split the work across two machines:

- **Development machine** (no dedicated GPU): all code, the full data pipeline, small CPU-only smoke tests on real (short) examples, and all post-hoc analysis that doesn't need to touch the model again.
- **RTX 4060 laptop**: the actual multi-hour training run, and running `evaluate.py` once against the finished checkpoint.

Code syncs between them through a private GitHub repository. Raw data, tokenized caches, and model checkpoints are `.gitignore` d, since they are too large and cheaply regenerable on whichever machine needs them. Result files (`metrics.json` , the raw prediction dump) are deliberately tracked in git instead, since those represent hours of GPU compute that shouldn't need to be reproduced to be seen.

5.2 Repository Structure

```
lettucedetect-reimpl/
├── docs/                          # this report, literature survey, project brief, original paper
├── data/
│   ├── raw/                       # RAGTruth's source_info.jsonl + response.jsonl (gitignored)
│   └── processed/                 # tokenized, label-aligned cache (gitignored)
├── src/
│   ├── data_prep.py                # load RAGTruth, align spans to token labels
│   ├── verify_alignment.py         # hand-verification of label alignment
│   ├── model.py                    # ModernBERT + token classification head
│   ├── dataset.py                  # cached-data → HF Dataset wrapper
│   ├── train.py                    # training loop (HF Trainer)
│   ├── evaluate.py                 # example-level + span-level F1
│   └── tune_threshold.py           # post-hoc threshold analysis
├── configs/base_config.yaml        # hyperparameters, resolved relative to repo root
├── checkpoints/                    # saved weights (gitignored)
└── results/                        # metrics.json, raw prediction dump (tracked)
```

5.3 Data Pipeline

RAGTruth's raw format (from ParticleMedia/RAGTruth) is two JSONL files joined by `source_id` : `source_info.jsonl` (2,965 rows, one per task/question/document) and `response.jsonl` (17,790 rows, up to 6 LLM answers per source item, from `gpt-4-0613` , `gpt-3.5-turbo-0613` , `mistral-7B-instruct` , and three Llama-2 chat variants, each with human-annotated hallucination spans as character offsets into the answer).

A real correctness decision came from inspecting this data directly rather than trusting the paper's prose description. `source_info` has a **different shape per task type**: for QA it's a dict with separate `question` and `passages` fields, while for Summarization and Data-to-text it's just the context, with no separate question field. Rather than force an inconsistent three-way split, we verified against the **official LettuceDetect reference implementation** (KRLabsOrg/LettuceDetect) and confirmed it uses the entire original `prompt` field, instructions, question, and passages, exactly as shown to the generating LLM, as one context block, paired with the answer via the tokenizer's built-in sentence-pair encoding. We adopted the same approach.

Label alignment (`tokenize_and_align_labels` in `data_prep.py`) is the step the project brief specifically flags as the highest-risk part of the whole pipeline, since a subtle bug here trains a model to completion with no error thrown, just meaningless labels. Key design points, all verified against the official reference code before being trusted:

- The tokenizer is called on `(context, answer)` as a pair, with `truncation="only_first"` . That guarantees the *context* gets cut before the answer ever does, so no answer token is ever lost to truncation.
- The answer's start token is located by **counting backward from the end** of the sequence (`total_length - answer_length - 1`), which stays correct regardless of how much context was truncated. Counting forward from the start would not.
- A token is labeled hallucinated (`1`) if its character range overlaps any annotated span at all (`token_end > span_start and token_start < span_end`); otherwise `0` . Context and special tokens are `-100` and excluded from the loss.

`verify_alignment.py` hand-checks this against real data before trusting it at scale: for 10 real RAGTruth examples that contain an annotated hallucination, it re-derives what text the pipeline's labels point to and compares it, character for character, against RAGTruth's own annotation. Result: **0 mismatches out of 10** (the only difference observed, an extra leading space on recovered spans, is expected tokenizer behavior, not an alignment error).

Final cache: 15,090 training examples and 2,700 test examples (summing to RAGTruth's full 17,790 responses, a useful consistency check on its own), with the test split evenly balanced at 900 examples each across QA / Summarization / Data-to-text.

5.4 Model and Training Pipeline

- **`model.py`** loads `answerdotai/ModernBERT-base` via `AutoModelForTokenClassification` with a 2-class head. The pretrained backbone's weights are used as-is; only the classification head is freshly initialized.
- **`dataset.py`** loads the cached, still-unpadded examples into a Hugging Face `Dataset`. Padding is deliberately deferred to batch time via `DataCollatorForTokenClassification`, so a batch of short examples never pays the compute cost of padding to the length of the dataset's longest example.
- **`train.py`** wraps this in a Hugging Face `Trainer`, configured from a YAML file (`configs/base_config.yaml`) whose paths are resolved against the repository root at runtime, so the same config file is valid unmodified on both machines. It includes a `--smoke-test` mode that trains on a handful of real (short) examples for one fast epoch, letting the whole pipeline be verified for runtime correctness on the CPU-only development machine before any GPU time is spent.

The CPU smoke test was run and passed before the first real training run: loss decreased over 10 steps, evaluation ran without error, confirming the tokenizer, collator, model, and Trainer were wired together correctly.

5.5 Hardware-Driven Deviations from the Paper

Setting	Paper	Ours	Reason
GPU	A100	RTX 4060 laptop (8GB)	Available hardware
Batch size	8	2, with gradient accumulation x4 (effective 8)	Avoids out-of-memory on 8GB VRAM while preserving the same effective batch size
Max sequence length	4,096	2,048	Reduces memory pressure; still covers the large majority of RAGTruth examples by length
Precision	not specified	bf16 (GPU only; full precision on CPU)	Reduces memory and speeds training on hardware that supports it

These are exactly the class of deviation the project brief anticipates and calls "expected, defensible scope decisions" rather than flaws, provided they're documented. That documentation is the purpose of this table.

5.6 Evaluation

`evaluate.py` runs the trained checkpoint over the full 2,700-example test set (inference only, no gradients) and computes:

- **Example-level F1**, the paper's headline metric: collapse each example's token predictions to one label (any predicted hallucinated token means the example is flagged), score with standard precision/recall/F1.
- **Span-level F1**, a metric we had to design ourselves, since neither RAGTruth nor the LettuceDetect paper ships a reference implementation. Ours works in token-index space: merge consecutive same-label tokens into spans, and count a predicted span as correct if it shares at least one token with some gold span (and symmetrically for recall). This is a reasonable, documented, but non-canonical choice. The number is not directly comparable to the paper's own self-implemented span metric, only informative about relative model quality.

Both metrics are also broken down **per task type** (QA / Summary / Data-to-text), matching how the paper's own Table 2 is structured.

To support further analysis without needing the GPU again, `evaluate.py` also dumps every answer token's raw predicted probability to `results/test_predictions.npz`, a few hundred KB against the checkpoint's hundreds of MB, so any future question about this specific trained model can be answered offline.

5.7 Threshold Analysis

The paper (and our initial evaluation) uses a fixed decision threshold of 0.5 with no justification beyond convention. **`tune_threshold.py`** investigates whether a different cutoff does better, using the raw probability dump from section 5.6, with no retraining and no GPU required.

The honesty concern this script exists to solve: picking whichever threshold maximizes F1 *on the test set*, then reporting that same F1, would silently turn the test set into something a parameter was fit to. The fix is to split the **test set itself** into two halves, grouped by `source_id` (RAGTruth has around 6 responses sharing one source document, so a naive random split could leak the same document into both halves). Tune the threshold on one half; report the resulting score only on the other, untouched half.

6. Results

6.1 Headline Result

Metric	Paper (LettuceDetect-base)	Our reimplementation
Example-level F1	**76.07**	**77.18**
Precision	not reported	80.4
Recall	not reported	74.2

Our independently written reimplementation matched, and slightly exceeded, the paper's published headline number.

6.2 Per-Task Breakdown

Task	Precision	Recall	F1	n (test)
Data-to-text	88.9	85.8	**87.3**	900
QA	65.5	68.8	**67.1**	900
Summarization	64.6	45.6	**53.4**	900

Summarization is the weakest task, driven by low recall specifically: the model misses more real hallucinations there than it false-alarms. **This matches the original paper's own reported pattern** of weaker summarization performance. Reproducing the same *qualitative* weakness the original authors found is stronger evidence of a faithful reimplementation than the aggregate F1 number alone, since a bug that happened to produce a plausible overall score would have no particular reason to also reproduce this specific per-task shape.

6.3 Span-Level Result

Span-level F1: 64.3 (precision 59.7, recall 69.7). Not directly comparable to the paper's own figure (see section 5.6), but well above chance, indicating the model localizes *which part* of an answer is unsupported, not just *whether* it is.

6.4 Threshold Sensitivity

Full sweep on the complete test set (sensitivity analysis only, not the reported result, since it was computed by looking at all of test):

Threshold	Example F1	Span F1
0.30	76.6	58.2
0.40	**77.8** *(peak)*	61.3
0.50 *(default)*	77.2	64.2
0.60	76.9	66.4
0.70	74.7	**66.6** *(near-peak)*

Honest, held-out result :

Threshold	Example F1
0.5 (default)	78.18
0.40 (tuned)	78.29

Finding: a single global threshold does not meaningfully move example-level F1. The gain (+0.11) is smaller than the noise you'd expect from simply re-measuring the same model on a different random half of the test set. It does, however, **meaningfully improve span-level F1** (64.2 to 66.6 moving from 0.5 to roughly 0.65 to 0.70): a stricter cutoff filters out isolated, low-confidence false-positive tokens that cost span-precision without being the tokens that decide example-level outcomes in the first place. This is a real, explainable, reportable finding, just not the "free F1 point" a naive threshold sweep might have suggested.

But tuning the threshold separately per task tells a different story. The global result above averages over all three tasks together, which hides a real, larger effect specific to the weakest one. Repeating the exact same honest held-out protocol (tune on one source-grouped half of that task's examples, report on the other), but stratified by task:

Task	F1 @ 0.5	F1 @ tuned threshold	Gain
Summary	53.4	**57.8** (threshold 0.40)	**+4.4**
QA	67.1	68.8 (threshold 0.45)	+1.7
Data2txt	87.3	87.5 (threshold 0.40)	+0.2

Summarization, our weakest task (section 6.2), gets by far the largest gain: +4.4 F1 points, for zero additional training. This makes sense once its own token composition is examined: hallucinated tokens make up only **3.0%** of Summary's answer tokens, against **8.0%** for QA and **4.8%** for Data2txt, the sparsest positive-class signal of the three tasks. A model trained under that scarcity tends to play conservative and only flag what it's very confident about, which is exactly the high-precision (64.6), low-recall (45.6) pattern reported in section 6.2. A lower, task-specific threshold directly compensates for that conservatism. We also checked and ruled out a competing hypothesis before proposing this: Summary examples are not disproportionately truncated by the 2,048-token cap (only 2.5% hit the limit, average length 868 tokens), so context loss is not the driver.

Deploying a per-task threshold is practical, not just a reporting trick: the task type is known at inference time in a real RAG pipeline, so applying a different cutoff per task is a normal, legitimate engineering choice, not something that requires guessing which task an answer belongs to.

6.5 Methodological Integrity Findings

Two things we found by auditing our own process, not by anyone external flagging them:

1. **A validation leak in checkpoint selection.** Our training configuration originally selected the "best" checkpoint by evaluating against the test set after every epoch, the same set the final number is reported against. We chose not to retrain with a proper held-out validation split, given the time/compute cost against the likely size of the effect (selecting among 6 checkpoints is a much smaller leak than tuning many hyperparameters against test), but we disclose it rather than presenting 77.18 as beyond

question. This is, notably, a small-scale instance of exactly the failure mode Dubanowska et al. (2025) warn the entire encoder-probe detector family is prone to (section 3), which made auditing for it here feel directly motivated by the literature we were citing, not an afterthought.

2. **Code provenance.** The tokenization and label-alignment strategy (section 5.3) was verified against the official KRLabsOrg/LettuceDetect reference implementation before being trusted on the full dataset. Everything else, including the model wrapper's usage, the full training pipeline and configuration system, both evaluation metrics (including the span-level metric, designed from scratch), and the threshold-tuning protocol, was independently written.

7. What Worked

- **The core reimplementation.** Built from scratch (own tokenization, own label alignment, own training loop, own metrics, not the authors' `pip install lettucedetect` package), it matched the paper's published result.
- **Hand-verification of label alignment before scaling up.** Catching a silent labeling bug is only possible by looking at real examples. Doing this before the first full training run meant the eventual multi-hour GPU run was trustworthy the first time.
- **A CPU smoke-test step before every GPU run.** Every runtime bug (a wrong field name, a shape mismatch) surfaced on a handful of examples in seconds on a machine with no GPU, rather than an hour into a real run on the machine that actually costs something to use.
- **Reproducing the paper's own reported weakness (summarization).** This is a stronger correctness signal than the headline number matching, since it's a pattern a bug would have no reason to reproduce.
- **An honest held-out protocol for threshold tuning**, rather than reporting whichever cutoff scored highest on test.
- **Dumping raw predictions once**, so every subsequent question about the trained model (threshold sweeps, this report's tables, future calibration work) could be answered without going back to the GPU.
- **Stratifying the threshold analysis by task instead of stopping at the global average.** This surfaced a real +4.4 F1 gain on our weakest task (Summarization, section 6.4) that the global number had hidden, for no additional training cost.

8. What Didn't Work / Challenges Encountered

- **Initial GPU memory constraints.** Batch size 8 (the paper's setting) did not fit comfortably in the RTX 4060's 8GB VRAM. Resolved with batch size 2 and gradient accumulation of 4, preserving the same effective batch size at some training-speed cost. Documented in section 5.5 as an expected hardware deviation, not a flaw.

- **A validation leak in our own checkpoint-selection setup**, found during our own methodological review rather than by an external reviewer (section 6.5). Not retrained, for cost/time reasons, but disclosed rather than hidden.

- **Threshold tuning produced a smaller effect than initially hoped.** The headline metric barely moved (+0.11 F1, noise-level). The honest result here is that the paper's default threshold was already close to optimal for this metric, which is a legitimate finding, just not the improvement we set out hoping to report. The compensating positive: this same investigation surfaced a real, different, explainable finding about span-level quality instead.

- **The combined RAGTruth+PsiloQA training study is not yet executed** as of this report. It is fully scoped (section 7.2 of the literature survey) but pending the next phase of work.

9. Conclusions

1. **The reimplementation is validated.** 77.18 example-level F1 against a published target of 76.07, from an independently written codebase, is a successful reproduction by the standard the project brief sets.
2. **Reproducing the paper's per-task weakness pattern, not just its aggregate score, is meaningful evidence of correctness.** A bug that happened to produce a plausible overall number would have no reason to also reproduce this specific shape.
3. **The paper's fixed 0.5 threshold is close to optimal for example-level F1**, at least for this trained model. The honest, held-out gain from tuning it is statistically indistinguishable from noise. It is *not* close to optimal for span-level localization quality, where a stricter threshold does meaningfully better by filtering spurious single-token false positives.

4. **Auditing one's own methodology is worth doing and reporting**, not just applying to the paper under review. Finding and disclosing our own validation leak, and building an honest tune/report split for threshold analysis, directly practices the same scrutiny the 2025 literature (Dubanowska et al.) argues this entire class of detector needs. That is a stronger position to present than a number with no methodology behind it.
5. **Checking whether a "novel idea" has already been published is worth the time it costs.** It surfaced a stronger, still-open contribution (combined-dataset training) in place of one that would have read as a repeated experiment.

10. Limitations

- **English-only, RAGTruth-only results** (as of this report). The combined-dataset, cross-lingual generalization question is scoped but not yet executed.
- **The validation leak in checkpoint selection was disclosed, not fixed.** The reported 77.18 may be a few tenths of a point optimistic versus a fully leak-free protocol. A retrain with a proper held-out validation split (grouped by `source_id`) would close this gap but was not done given time/compute constraints.
- **Span-level F1 is not paper-comparable**, by design of the problem (no canonical metric exists). It is only meaningful as a relative, within-project measure.
- **ModernBERT-base only.** ModernBERT-large (the paper's stronger, 396M-parameter variant, 79.22 F1) was not attempted, due to the additional VRAM and training time it would require on the available hardware.
- **Max sequence length capped at 2,048**, not the paper's 4,096, for memory reasons. This is documented in section 5.5, but it means a small fraction of longer RAGTruth examples are truncated more aggressively than in the original paper.
- **The combined RAGTruth+PsiloQA training study, the project's actual "beyond the paper" contribution, is pending**, not completed, as of this report.

11. Future Work

Directly continuing the plan in the project brief and the literature survey:

- **Immediate next step:** execute the combined-training study (section 7.2 of the literature survey). Train on RAGTruth, on PsiloQA-English, and on both combined; evaluate each configuration on both test sets; report calibration (Expected Calibration Error) alongside F1 for all three.
- **Class-weighted retraining, targeted specifically at Summarization's recall gap.** The per-task threshold result (section 6.4) is a free, already-realized gain, but it only compensates after the fact for a model that was trained under a scarce positive-class signal (3.0% of Summary's answer tokens are hallucinated, against 8.0% for QA). Reweighting the loss to penalize missed hallucinated tokens more heavily during training targets that scarcity at its source rather than adjusting for it afterward, and could plausibly lift Summary's recall (and F1) further than thresholding alone. This is a genuine "planned, not yet executed" item: it requires a real retrain, and unlike the threshold result above, we are not claiming it will work until it has actually been run.

12. References

See `literature_survey_and_presentation.md` section 9 for the complete, annotated reference list. Primary sources for this report:

1. Kovács, Á. & Recski, G. (2025). *LettuceDetect: A Hallucination Detection Framework for RAG Applications.* arXiv:2502.17125
2. Niu, C. et al. (2024). *RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Generation Systems.* ACL 2024. Data: github.com/ParticleMedia/RAGTruth
3. Official reference implementation: github.com/KRLabsOrg/LettuceDetect. Consulted to verify the tokenization/label-alignment strategy in section 5.3.
4. Dubanowska, K. et al. (2025). *Representation-based Broad Hallucination Detectors Fail to Generalize Out of Distribution.* EMNLP 2025 Findings.
5. (2025). *When Models Lie, We Learn: Multilingual Span-Level Hallucination Detection with PsiloQA.* arXiv:2510.04849, EMNLP 2025 Findings.